

# Architetture e Sistemi Operativi

Enrico Melis

a.a. 2025-2026

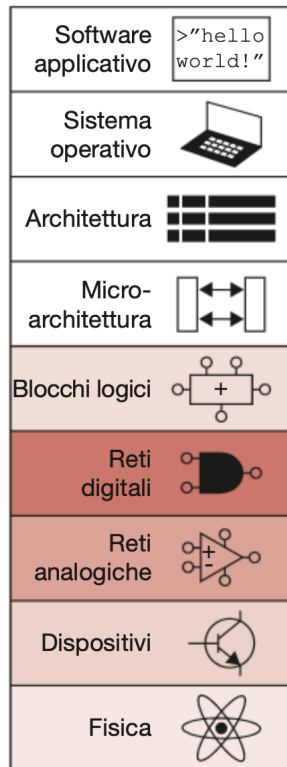
# Indice

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduzione al corso</b>                       | <b>3</b> |
| 1.1      | Livelli di astrazione . . . . .                    | 3        |
| 1.2      | Nozioni sui transistor . . . . .                   | 4        |
| 1.3      | Logica booleana . . . . .                          | 4        |
| <b>2</b> | <b>Reti Logiche</b>                                | <b>5</b> |
| 2.1      | Reti logiche combinatorie . . . . .                | 6        |
| 2.2      | Ottimizzazione nella logica combinatoria . . . . . | 9        |
| 2.3      | Verilog . . . . .                                  | 9        |
| 2.4      | Reti logiche sequenziali . . . . .                 | 9        |
| 2.5      | Componenti standard . . . . .                      | 9        |
| <b>3</b> | <b>Memorie</b>                                     | <b>9</b> |
| <b>4</b> | <b>Assembler</b>                                   | <b>9</b> |
| 4.1      | Istruzioni standard e utilizzo . . . . .           | 9        |
| 4.2      | Codifica della istruzioni . . . . .                | 9        |
| <b>5</b> | <b>Parallelismo</b>                                | <b>9</b> |
| <b>6</b> | <b>Microarchitetture</b>                           | <b>9</b> |
| 6.1      | Single cycle . . . . .                             | 9        |
| 6.2      | Multi cycle . . . . .                              | 9        |
| 6.3      | Pipeline . . . . .                                 | 9        |

# 1 Introduzione al corso

## 1.1 Livelli di astrazione

Come possiamo vedere anche nel libro [1], arriviamo ai sistemi operativi partendo da concetti molto più bassi.



Tralasciando la fisica e accennando la parte sui transistor, iniziamo il nostro cammino di astrazione con la reti logiche. I livelli di astrazione trattati sono (con alcuni esempi):

- Sistema Operativo (driver dei dispositivi)
- Architettura (istruzioni assembler, registri)
- Microarchitettura (datapath)
- Blocchi logici (sommatori, memorie)
- Reti digitali (gate logici AND, OR)

I livelli di astrazioni sono fondamentali per comprendere come funzionano tutte le componenti che diamo per scontato, costruendole man mano dal basso e incrementando la complessità.

## 1.2 Nozioni sui transistor

(In italiano vengono chiamati “transistor”, ma mi rifiuto). I transistor sono, al giorno d’oggi, l’unità di misura principale per gli effettivi calcoli all’interno dei calcolatori. I primi calcolatori sono stati costruiti con degli ingranaggi, adesso si utilizzano i transistor che si dividono in:

- BJT: *Bipolar Junction Transistors*
- MOS: *Metal-Oxide-Semiconductor Field Effect Transistor (MOSFET)*

I transistor MOS sono costruiti partendo dal silicio (Si, IV gruppo), che forma 4 legami covalenti. Per aumentare la conduttività vengono aggiunti degli atomi droganti, che possono essere del V gruppo (arsenico) o del III (boro). Dato che l’arsenico “ha un elettrone in più” e porta una carica negativa, viene chiamato drogante di tipo **n**, mentre il boro “porta una carica positiva”<sup>1</sup> e quindi viene chiamato drogante di tipo **p**.

**\*\* Work in progress \*\***

## 1.3 Logica booleana

La logica booleana<sup>2</sup> è quella sottosezione dell’algebra in cui le variabili possono avere solamente valori binari, cioè  $x \in \{0, 1\}$  oppure in senso astratto *vero* o *falso*. Le basi si fondano sui gate logici AND( $\wedge, \cdot$ ), OR( $\vee, +$ ), NOT( $\neg, \bar{x}$ ) che sono realizzabili tramite transistor.

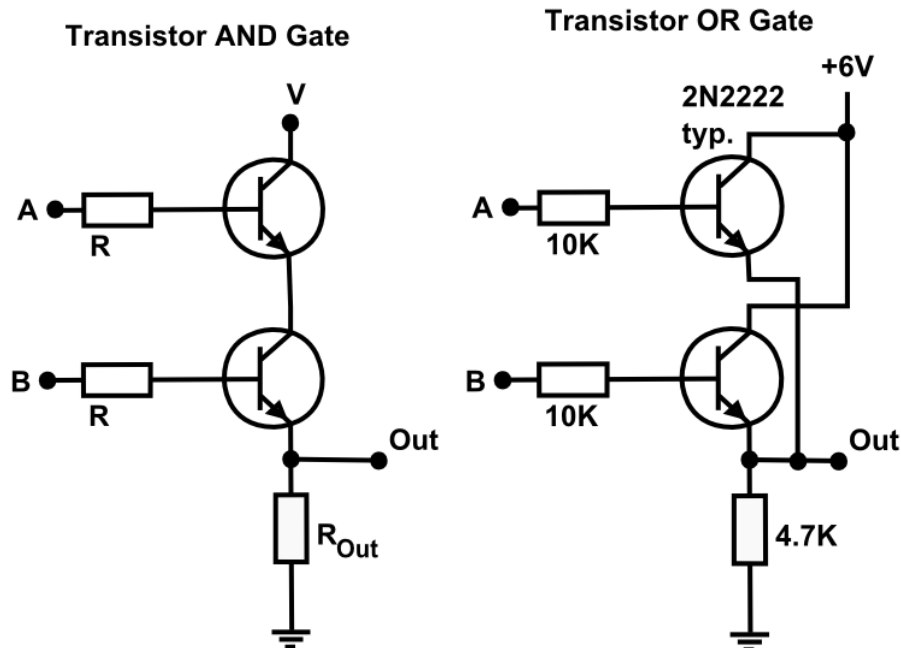
Queste sono le tabelle di verità per i gate logici appena menzionati:

| $p$ | $q$ | $p \cdot q$ | $p$ | $q$ | $p + q$ | $p$ | $\bar{p}$ |
|-----|-----|-------------|-----|-----|---------|-----|-----------|
| 0   | 0   | 0           | 0   | 0   | 0       | 0   | 1         |
| 0   | 1   | 0           | 0   | 1   | 1       | 1   | 0         |
| 1   | 0   | 0           | 1   | 0   | 1       |     |           |
| 1   | 1   | 1           | 1   | 1   | 1       |     |           |

Per avere una rappresentazione visiva di come sono realizzabili tramite transistor, ci basti pensare che per ottenere un AND basta mettere due transistor in sequenza, per l’OR basta metterli in parallelo e per il NOT basta invertire il segnale in input. Di seguito delle illustrazioni prese da Wikipedia.

<sup>1</sup>Spero che chimica, fisici e ingegneri elettronici mi risparmieranno per questo scempio che ho appena scritto però rende l’idea.

<sup>2</sup>George Boole (1815 – 1864) è stato un matematico e logico inglese.



### 1.3.1 Composizionalità

È importante tenere a mente che questi tre gate logici sono i veri gate fondamentali. Spesso si possono trovare altri gate categorizzati in questo modo ma lo sono soltanto per astrazione, dato che tutti i circuiti logici sono costruibili tramite questi 3. Ad esempio, il gate XOR è spesso considerato tra i fondamentali però non è altro che una composizione in formula dei precedenti tre già citati. La formula logica per lo XOR infatti è  $A \oplus B = ((A \wedge \neg B) \vee (\neg A \wedge B))$ . Capire questo è fondamentale sia per capire i livelli di astrazione (l'esempio dello XOR serve a capire quanto sia utilizzato e quanto possa essere utile astrarre la sua composizione per comodità) e il concetto di ritardo dei segnali. Ogni gate logico ha bisogno di un certo tempo  $\Delta t$  per stabilizzarsi e in base a come componiamo i vari gate questi tempi si sommano. La composizionalità è fondamentale anche per capire il concetto di *modularità*, fondamentale nella filosofia UNIX.

## 2 Reti Logiche

Le reti logiche sono dei circuiti che elaborano segnali logici booleani tramite la composizione dei gate logici fondamentali. Possono essere usate per calcolare funzioni o per memorizzare segnali.

## 2.1 Reti logiche combinatorie

Le reti logiche combinatorie modellano relazioni ingresso-uscita senza memoria: l'output dipende *solo* dagli input presenti. La progettazione segue un flusso standard:

1. Descrizione della funzione booleana
2. Costruzione della tabella di verità
3. Derivazione dell'espressione logica (forma SOP – *sum of products*)
4. Sintesi della rete di porte

Con “somma di prodotti” (SOP) si intende la scrittura della funzione come OR (somma) di termini AND (prodotti), derivati direttamente dalle righe della tabella di verità in cui l'uscita vale 1.

### Esempio: progettazione di un Full Adder a 1 bit

Il *Full Adder* è un circuito combinatorio che somma due bit  $x$  e  $y$  insieme a un riporto in ingresso  $c_{in}$ , producendo una somma  $z$  e un riporto in uscita  $c_{out}$ .

#### Specifica del problema.

- **Input:** bit  $x$ , bit  $y$ , riporto in ingresso  $c_{in}$
- **Output:** bit somma  $z$ , riporto in uscita  $c_{out}$

**Descrizione della funzione.** L'obiettivo è realizzare un sommatore completo a 1 bit: dati due bit da sommare e un eventuale riporto proveniente da una somma precedente (per composizionalità in somme multi-bit), il circuito deve calcolare il bit di somma e il nuovo riporto da propagare.

**Costruzione della tabella di verità.**

| $x$ | $y$ | $c_{in}$ | $z$ | $c_{out}$ |
|-----|-----|----------|-----|-----------|
| 0   | 0   | 0        | 0   | 0         |
| 0   | 0   | 1        | 1   | 0         |
| 0   | 1   | 0        | 1   | 0         |
| 0   | 1   | 1        | 0   | 1         |
| 1   | 0   | 0        | 1   | 0         |
| 1   | 0   | 1        | 0   | 1         |
| 1   | 1   | 0        | 0   | 1         |
| 1   | 1   | 1        | 1   | 1         |

**Derivazione della forma canonica SOP.** Per ottenere l'espressione booleana, individuiamo nella tabella di verità tutte le righe in cui l'uscita vale 1, separatamente per  $z$  e per  $c_{out}$ .

Per la somma  $z$ , l'uscita vale 1 nella seconda, terza, quinta e ottava riga. Da ciascuna riga costruiamo un *mintermine* (prodotto logico) delle variabili di ingresso: se un ingresso vale 0 usiamo la forma negata, se vale 1 la forma diretta. La somma (OR) di tutti i mintermini produce l'espressione per  $z$ :

$$z = \bar{x} \cdot \bar{y} \cdot c_{in} + \bar{x} \cdot y \cdot \bar{c}_{in} + x \cdot \bar{y} \cdot \bar{c}_{in} + x \cdot y \cdot c_{in} \quad (1)$$

L'Equazione 1 elenca tutte e sole le combinazioni di input per cui  $z = 1$ .

Procediamo analogamente per il riporto in uscita  $c_{out}$ , sommando i mintermini relativi alle righe in cui  $c_{out} = 1$ :

$$c_{out} = x \cdot y \cdot \bar{c}_{in} + x \cdot \bar{y} \cdot c_{in} + \bar{x} \cdot y \cdot c_{in} + x \cdot y \cdot c_{in} \quad (2)$$

**Sintesi della rete di porte.** A partire dalle espressioni SOP (Equazioni 1 e 2), la sintesi circuitale procede come segue:

1. Creare una linea verticale per ogni segnale di ingresso ( $x$ ,  $y$ ,  $c_{in}$ ) e le rispettive negazioni.
2. Realizzare ciascun prodotto con una porta AND a tre ingressi, collegando le linee corrette (dirette o negate).
3. Collegare le uscite delle porte AND agli ingressi di una porta OR per ciascun output.
4. L'uscita della porta OR corrisponde al segnale di output finale ( $z$  o  $c_{out}$ ).

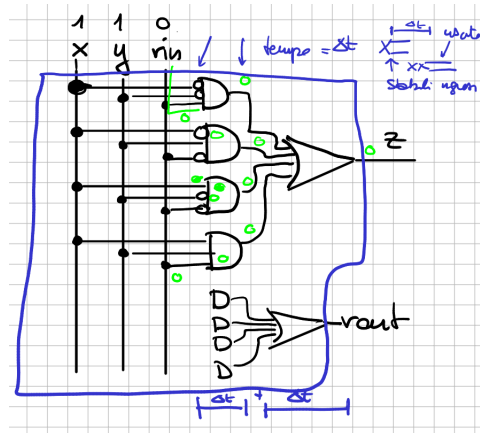


Figura 1: Implementazione circuitale del Full Adder a 1 bit: le linee verticali rappresentano i segnali di ingresso  $x$ ,  $y$  e  $c_{in}$  (con le relative negazioni); le porte AND realizzano i prodotti della forma SOP (Equazione 1 per la somma, Equazione 2 per il riporto); le porte OR combinano i prodotti per produrre le uscite finali  $z$  e  $c_{out}$ .



## 2.2 Ottimizzazione nella logica combinatoria

## 2.3 Verilog

## 2.4 Reti logiche sequenziali

### 2.4.1 Automi di Mealy

### 2.4.2 Automi di Moore

## 2.5 Componenti standard

### 2.5.1 Full Adder

### 2.5.2 Mux e Demux

### 2.5.3 Comparatore

### 2.5.4 Shifter

### 2.5.5 Encoder e Decoder

## 3 Memorie

## 4 Assembler

### 4.1 Istruzioni standard e utilizzo

### 4.2 Codifica della istruzioni

## 5 Parallelismo

## 6 Microarchitetture

### 6.1 Single cycle

### 6.2 Multi cycle

### 6.3 Pipeline

#### 6.3.1 Dipendenze

#### 6.3.2 Valutazione dei tempi di esecuzione

## Riferimenti bibliografici

- [1] Sarah L. Harris David Money Harris. *Sistemi digitali e architettura dei calcolatori*. Zanichelli, 2017.